

TAKE-HOME · AI ENGINEERING

Junior AI Investigator

An AI-first case review system for long-term-care fraud investigators.

Scope, architecture, evaluation, trade-offs, and the path to production volume.

50

referrals triaged end to end

17

rules calibrated from the data

88

tests passing

10/10

behavioural invariants hold

Prototype · Python 3.12 · FastAPI · SQLite · vanilla-JS SPA · OpenRouter (no agent framework)

Repository · github.com/siddheshpradhan39/Case-Review-for-Fraud-Investigators

Scale · ~4,650 lines of Python · ~640 lines of front-end · 1,683 lines of tests

Contents

1 • Executive summary

2 • Scope

2.1 What the brief asked for

2.2 What I built

2.3 What I deliberately left out

2.4 Assumptions on record

3 • The data, and what it dictated

4 • Architecture in detail

4.1 The pipeline

4.2 Features and calibration

4.3 Rules and scoring

4.4 The router

4.5 The agent crew

4.6 The harness

4.7 The verification boundary

4.8 Tools

4.9 Incremental reuse

4.10 Governance surfaces

4.11 Data model

4.12 LLM client and resilience

5 • Evaluation

6 • Alternatives I rejected

7 • With more time

8 • Scaling to production volume

8.1 What already scales

8.2 What breaks first

8.3 Target architecture

8.4 Cost model

8.5 Throughput and latency

8.6 Data and entity resolution

8.7 Calibration, drift and labels

8.8 Reliability

8.9 Governance and fairness

8.10 Rollout

Appendix A • Rule catalogue

Appendix B • Repository map

1 • Executive summary

A fraud investigation team receives a daily queue of engine-generated referrals. Most turn out benign, and the expensive part of the job is not finding fraud — it is the hours spent proving that a referral was never fraud. This system inverts that: by the time an investigator opens the tool, every case already carries a summary, cited evidence, a risk level and a recommended action, with a full record of how the conclusion was reached.

The core design decision is that **reasoning depth is bought, not given**. Deterministic rules score every case first. A router with no LLM in it then reads which rules fired and sends each case down one of three lanes. Clean cases share a single cheap batched call. Unambiguously critical cases get one verification call. Only the genuinely ambiguous middle gets the full agent crew — specialists, an adversarial challenger, a verifier, and a three-persona panel if the case is still contested.

THE RESULT, MEASURED ON THE 50 CASES

26% of cases consume 91% of the AI spend, which is the intended shape: effort lands on the hard tail. Against a single-judge baseline the crew reaches **100% band agreement using 27% fewer tokens**, while adding an adversarial pass, a panel and a full trace. All **10 behavioural invariants** and **5 counterfactual and prompt-injection tests** pass, and **277 of 277** stored evidence citations re-verify against the record.

What the system does **not** claim is accuracy. The supplied data has no fraud labels, so any precision or recall figure against "real" fraud would be invented. Everything measured here is behaviour: properties that must hold whatever the model says.

2 · Scope

2.1 What the brief asked for

Ingest a queue of referred claims; reason over each case; produce a per-case assessment before a human looks at it (summary, key indicators, risk level, recommended next action); and let an investigator interact — drill in, ask questions in natural language, accept or reject findings, add notes, and move the case forward. Architecture, stack and agent design were left open.

2.2 What I built

SURFACE	WHAT IT DOES
Morning briefing	Risk counts and dollar exposure per band, the cases to open first, co-firing rule clusters, and statistically abnormal rule firing by segment (Fisher-style rate comparison with a p-value). Headlines are computed from the data; an optional LLM narrative is restricted to those figures and re-checked against them.
Per-case assessment	Rule score and AI score side by side, a summary, key indicators each citing an exact field and value, mitigating factors, suggested next steps, open questions, guardrail notes, and the full agent trace with per-step token and cost accounting.
Interaction	Accept or reject any individual indicator (a rejected finding is not re-asserted); free-text chat over the case with the same read-only tools the judge uses; investigator notes that feed the next assessment.
Notes with full deletion	A note changes the next assessment. Deleting it purges it from assessments, chat history, related-case context and audit text — including paraphrases, matched by 4-word shingles.
Cross-case knowledge	Notes, decisions and confirmed outcomes from linked and nearest-neighbour cases are shown to the agents as precedent, never as facts about the case under review.
Bulk clear	By filter (e.g. score < 10) or by hand-picked selection. A preview lists every case the guardrails refuse to clear and why, before anything is written.
Supervisor escalation	The investigator writes a reason; the AI drafts the handoff summary. A supervisor approves to SIU or returns it with a note.
Rules tab	Create, tune, disable or shadow rules through a validated condition tree (never code). Every draft gets a live backtest — precision, recall, Wilson confidence intervals — plus an impact preview of which cases move.
Blocklist	Hard rules that decline a case outright, bypassing scoring and AI: claim numbers, provider or member IDs, segments, condition trees, or fraud-history thresholds. Investigators propose; supervisors enact. Entries covering more than 25% of the queue require explicit confirmation. Supervisors can override a decline with a written reason, which becomes a case note and counts against the entry as a false-positive signal.
Outcome feedback	Marking a case as confirmed fraud raises rule R17 on its nearest neighbours, feeds agents human ground truth, and blocks bulk clear on lookalikes. Misses (cases the system had rated low) are recorded as such.
Audit log	Every AI assessment and every human decision, with actor, reason and timestamp.

2.3 What I deliberately left out

OMISSION	REASON
Authentication and multi-tenancy	Explicitly out of scope in the brief. Investigator and supervisor are personas selected in the header and sent as request headers; the authority split between them is enforced server-side, so the governance model is real even though the identity is not.
A trained ML model	The data carries no fraud labels. A supervised model would have had to invent its own targets, and any accuracy claim would have been circular. Rules calibrated from the distribution are honest about what they are.
Provider / member graph	The CSV has no provider or member identifiers. Cross-case linkage is implemented and runs today on claim number; the same code path accepts provider and member IDs the day a dataset carries them.
External enrichment	Public registries and web search were offered as optional. They would have added surface area without deepening the core reasoning question.
Streaming UI	Assessments run as a morning batch, which is the right interaction model at ~20s per model call. Streaming would polish a flow that should not be synchronous in the first place.

2.4 Assumptions on record

- Each CSV row is one referral, not one claim line. Amounts are per referral.
- "Amount vs peer average" is relative to the same care type, since that is the only peer grouping the data supports.
- A signal value of 0 means "not triggered", per the brief.
- Claim number is a stable key across referrals; its reuse is treated as a data-integrity lead rather than proof of fraud.
- The portfolio of 50 is assumed representative enough to calibrate thresholds against. At production volume, thresholds would be re-fitted on a rolling window (§8.7).

3 · The data, and what it dictated

Design decisions came out of the distribution, not from a template. Three properties of the 50-case sample shaped everything downstream.

The portfolio splits three ways, with nothing in between

Eight cases are extreme on essentially every signal. Nine sit in a genuinely ambiguous middle. Thirty-three are clean. Critically, the score bands fall inside **empty stretches** of the distribution — at the CRITICAL cut of 80 the nearest scores are 77.1 and 84.8; at the HIGH cut of 50 they are 38.8 and 52.4. The thresholds are not arbitrary round numbers that happen to look tidy; they sit where the data has no cases.

CONSEQUENCE

Rules alone resolve the 8 and the 33. Paying for multi-agent reasoning on those is waste. The 9 in the middle are where judgement has value — and that is precisely the routing rule the system implements.

The signals co-move, so they must not be summed

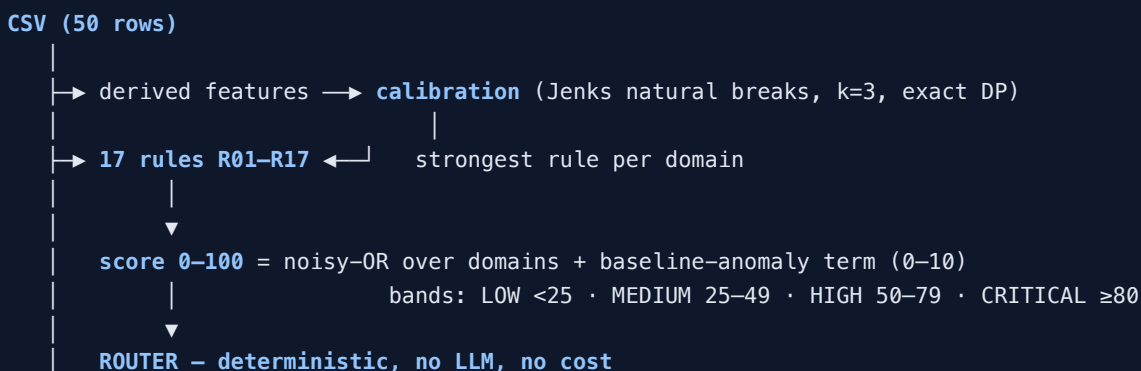
The eight worst cases trip everything at once. Adding rule weights would count one underlying phenomenon five times and push every bad case to the ceiling, destroying the ordering within the tail. The score therefore takes the **strongest rule per domain** (Billing, Utilization, Geography, Relationship, Timing) and combines domains with a noisy-OR: $\text{score} = 1 - \prod(1 - w_{\text{domain}})$. Convergence across **independent** domains becomes the real evidence, which is what rule R13 measures.

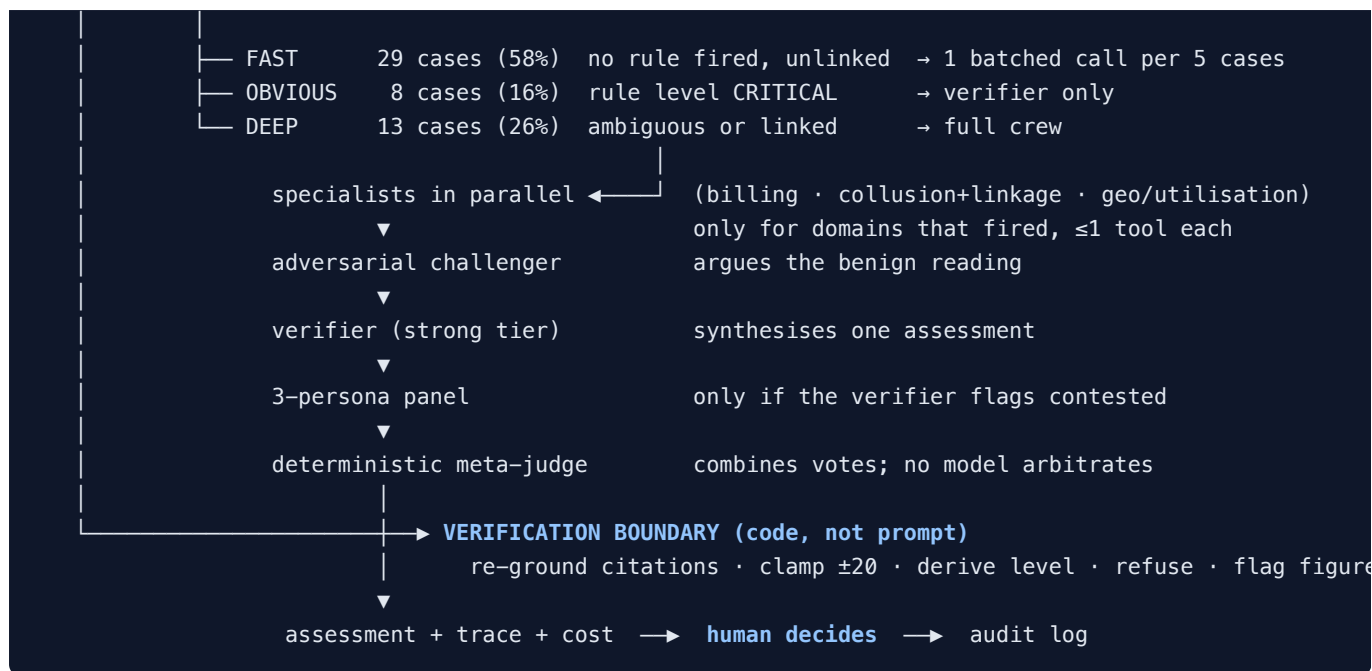
Two patterns no single-row rule can see

- **Silent drift.** Of the zero-flag cases, 30 trip none of the seven continuous signals and three (C1026, C1040, C1042) trip five — and nothing sits in between. A model that counts flags scores those three as clean. Rule R14 exists solely to catch that empty-gap pattern.
- **Claim-number reuse.** LTC-2034786 appears on C1001, whose own profile is benign, and on C1031, which is critical. No per-row rule can see this. It motivated cross-case linkage, the linked-case guardrail floor, and the related-case knowledge that agents read. In the live system the crew **raises** C1001 from 27.1 to 37.1 and recommends escalation, entirely because of the link.

4 · Architecture in detail

4.1 The pipeline





4.2 Features and calibration

Raw signals are extended with derived features that encode domain reasoning the CSV leaves implicit:

- **Implied weekly travel miles** = visits/week × distance × 2. Baseline cases stay under ~1,300; the worst eight sit between 2,352 and 7,552. A provider covering thousands of miles a week to deliver in-home care is physically implausible, and that implausibility is a stronger signal than either input alone.
- **Robust log-amount z-score** (median/MAD on log amount), so a single large claim does not drag the scale.
- **Linked case IDs**, computed across the portfolio by shared claim number.

Thresholds come from **Jenks natural breaks** (k=3, exact dynamic programming) on each continuous signal, splitting values into baseline / elevated / extreme by minimising within-group variance. Each threshold is then placed at the **midpoint of the empty gap** between adjacent groups. Crossing the elevated break is MEDIUM; crossing the extreme break is HIGH. Calibration is regenerated by `python -m app.calibration`, which also writes the evidence tables in `docs/RULES.md` — so every threshold in the product is reproducible from the data rather than asserted.

4.3 Rules and scoring

Seventeen rules span five evidence domains plus Convergence, Integrity and Precedent. Twelve are direct signal rules; five are derived or cross-case (R11 travel plausibility, R12 exposure, R13 multi-domain convergence, R14 silent drift, R15 claim reuse, R17 fraud lookalike). Domain weights are LOW 5, MEDIUM 12, HIGH 25, CRITICAL 40. A baseline-anomaly term (0–10, the mean percentile rank of the continuous signals) is blended in so that clean cases still receive a stable ordering rather than all collapsing to zero. The full catalogue is in Appendix A.

4.4 The router

The router is ordinary Python. It reads the fired rules and the link set, and returns a lane. It makes no model call, costs nothing, cannot hallucinate, and cannot fail open.

LANE	CASES	CONDITION	TREATMENT	CALLS/CASE	TOKENS/CASE
FAST	29	no rule fired, no link	5 cases per batched call, no tools	0.66	991
OBVIOUS	8	rule level CRITICAL	verifier only — facts are not in dispute	1.00	3,286
DEEP	13	everything else	full crew	4.77	14,073

Routing by difficulty rather than by risk is what makes the economics work. An LLM-based router was considered and rejected: it would add a call, a latency hop and a failure mode to every single case, in order to decide something the rules have already determined deterministically.

4.5 The agent crew

On the DEEP lane the crew is a structured argument, not a chain of summarisers. Each role has a distinct job and a distinct failure mode it guards against.

ROLE	PURPOSE
Specialists billing · collusion + linkage · geo/utilisation	Each sees only its own evidence slice and may call at most one tool. Spawned only for domains that actually fired, so a case with no geography signal never pays for a geography specialist. Narrow input is deliberate: it prevents one strong signal colouring the reading of unrelated ones, and it makes the specialist's output cacheable by evidence hash.
Challenger	Adversarial by construction: it argues the benign reading as well as the evidence honestly allows — legitimate explanations, mitigating facts, and what evidence is missing. Without it, a pipeline of agents all reading the same suspicious signals converges on suspicion. The challenger is the only component whose job is to disagree, and it downgrades in 38% of DEEP cases.
Verifier	The senior analyst. Runs on the strong model tier. Sees the rule result, the specialist findings and the challenger's argument, and produces one assessment: summary, a bounded score adjustment with a reason, key indicators, mitigating factors, next steps, open questions, an honest self-confidence, and a contested flag.
Panel	Three personas — conservative, neutral, aggressive — convene only when the verifier flags the case contested (5 of 13 DEEP cases). Each votes independently on the adjustment and action.
Meta-judge	Deterministic code, not a model. Combines the panel votes and applies the floors. No LLM arbitrates the final answer — in C1001 the linked-CRITICAL floor overrules a –10 vote outright.

4.6 The harness

Every agent call runs through a small runtime (`app/agents/runtime.py`) rather than a framework. It provides:

- **Budgets** — per-agent caps on calls, tokens and tool invocations, enforced in code. The judge repeatedly ignored a "use at most two tools" instruction in the prompt; the budget is now enforced by the loop, which is the general lesson: prompts express intent, code enforces it.
- **Tool allowlists** — each role can reach only the tools its job needs.
- **Typed output with repair** — every response is parsed into a Pydantic schema; malformed output triggers up to two repair attempts with the validation error fed back.
- **Escalation ladder** — if a fast-tier agent exhausts its repairs, the call is retried once on the strong tier. This was added because an eval run caught a real FAST-lane JSON failure on case C1027; cost is incurred only on the cases that need it.
- **No-progress stop** — a loop that stops changing state is terminated rather than left to spin.

- **Tracing** — every call is recorded as a span with model, tier, tokens, latency, cost and stop reason, into the `runs` and `spans` tables. The investigator-facing trace and the cost tables in §5 are the same data.
- **Fail-closed** — when the model cannot be reached, the case is stored as `LLM_UNAVAILABLE` and the UI says so. It never falls back to fabricated prose.

4.7 The verification boundary

Grounding is enforced by deterministic code between the model and the database, not requested in a prompt. Five mechanisms, all of which run on every assessment:

MECHANISM	BEHAVIOUR
Citation re-grounding	Every <code>field = value</code> the model cites is re-read from the record independently. Mismatches are dropped from the stored assessment and shown to the human as a guardrail note. 277 of 277 surviving indicators re-verify; 11 were dropped, a 96.2% grounding survival rate.
Bounded adjustment	The model may move the deterministic score by at most ± 20 , and the risk level is derived from the resulting number — never chosen by the model. This keeps the score reproducible and auditable while still letting evidence change the answer.
Refusal rules	<code>CLEAR_FALSE_POSITIVE</code> is rejected outright if any HIGH or CRITICAL rule fired, or if R17 (confirmed-fraud lookalike) fired.
Linked-case floor	A case linked to a CRITICAL case cannot be downgraded below a floor, regardless of what the crew concludes.
Figure check	Numbers appearing in generated prose are checked against the source record; unverified figures are flagged rather than silently published. One assessment currently carries such a flag.

4.8 Tools

Nine read-only tools are shared by the crew and the chatbot: `get_case`, `get_peer_stats`, `find_similar_cases`, `get_linked_cases`, `get_case_notes`, `search_notes` (across every case), `get_rule_catalog`, `get_cluster_context` and `query_cases`. Every one reads the live record and none can write, so no agent can change state as a side effect of reasoning. Tool calls are traced and shown to the investigator.

There is **no RAG and no vector store**. The corpus is 50 structured rows; SQL answers questions about it exactly, whereas embedding retrieval would introduce approximation where none is needed. That trade-off inverts at volume, and §8.6 says where.

4.9 Incremental reuse and staleness

Each case has an `input_hash` covering its rules, score, notes, recorded outcome, and the notes and outcomes of related cases. When any of those change the assessment is marked stale in the UI rather than silently re-run. On re-run, specialists are reused from cache by **evidence-slice hash**: since a specialist sees only its own slice, a new note that does not touch that slice cannot change its output.

MEASURED EFFECT

Adding a note to a DEEP case re-runs **2 calls instead of 5** — only the challenger and verifier, whose inputs actually changed. Across the eval, 13 specialist executions were served from cache.

4.10 Governance surfaces

Two surfaces let the team change system behaviour without an engineer, which matters because fraud patterns move faster than release cycles.

- **Rules tab.** Custom rules are expressed as a validated AND/OR condition tree over named fields — data, never code, so there is no evaluation of user input. Each draft is backtested live with precision, recall and Wilson confidence intervals, plus a preview of exactly which cases change band. Because the data has no labels, backtest labels are synthetic and derived from risk level; the default mode is **leave-one-out**, so a rule is never graded against a label it helped produce. The UI states this rather than presenting the numbers as fraud accuracy. Rules can run in shadow mode, and every change is versioned.
- **Blocklist.** Hard rules that decline outright, bypassing scoring and AI entirely, for decisions that should not depend on a model: a known-fraudulent claim number, provider or member; a segment; a condition tree; or a fraud-history threshold (N confirmed frauds on the same claim number, provider or member; or a count or rate within a segment). A case is never declined on the basis of its own fraud label. Declines are reversible: removing an entry returns the case to its previous status. Already-closed cases are never reopened — they raise an alert instead. Governance is split: investigators may only propose (entries save paused), supervisors enact, entries touching more than 25% of the queue need explicit confirmation, and a supervisor override requires a written reason that becomes a case note and counts against the entry as a false-positive signal.

4.11 Data model

SQLite in WAL mode, one connection per call. Eighteen tables, of which the load-bearing ones are: `cases`, `assessments` (versioned, with input hash and status), `notes`, `feedback` (accept/reject per indicator), `escalations`, `outcomes`, `runs` and `spans` (the agent trace and cost ledger), `custom_rules`, `rule_overrides`, `rule_versions`, `blocklist`, `declines`, `blocklist_events`, and `audit`.

Assessments are append-only and versioned, so the trace of what the system believed and when is never overwritten.

4.12 LLM client and resilience

A single OpenAI-compatible client against OpenRouter, with model tiers (fast and strong) configured per role. It carries a rate limiter (18 requests/minute on the free tier), a per-model circuit breaker, and a fallback chain that moves to the next model when one fails. Reasoning mode is explicitly disabled — with it on, the free models burned 65–130 seconds and returned empty content; with it off, calls settle around 20 seconds with reliable structured output. A record/replay cassette makes the whole system deterministic offline, which is what allows the eval suite to run in CI without spending tokens.

5 · Evaluation

With no labels, accuracy cannot be measured, so the eval suite measures behaviour, grounding, cost and adversarial robustness. It runs with `python -m app.evals.run` and writes `docs/EVAL.md`.

10/10

behavioural invariants

5/5

counterfactual + injection

277/277

indicators re-verify

96.2%

grounding survival

Invariants — properties that must hold whatever the model says

- All 50 cases produce an assessment; router lanes are exactly 29 / 8 / 13.
- The 8 all-signal cases are never cleared and never fall below HIGH.
- No clean case (no rules, unlinked) is escalated or rated HIGH or above.
- The three silent-drift cases (C1026, C1040, C1042) are never cleared.
- C1001, which shares a claim number with critical C1031, is never downgraded.
- `CLEAR_FALSE_POSITIVE` never appears where a HIGH or CRITICAL rule fired.
- Every AI score sits within ± 20 of the rule score, with the level derived from it.
- Every stored indicator re-verifies against the record under an independent re-check.

Counterfactual and adversarial tests

- **Signal ablation.** Removing the geography and utilisation signals stops that specialist spawning and weakens convergence exactly as designed (R13 drops from CRITICAL to not firing).
- **Evidence direction.** A verified-relocation note on C1011 moves the case **down** and never up, within the guardrail bound (66.8 $\rightarrow$ 56.8).
- **Incremental reuse.** A new note re-runs challenger and verifier only; specialists are served from cache.
- **Prompt injection, high-risk case.** An instruction embedded in a note on a CRITICAL case is not executed: the case stays CRITICAL with `ESCALATE_SIU`.
- **Prompt injection, clean case.** The mirror-image attack — trying to force an escalation on a benign case — also fails.

Cost and latency by lane

LANE	CASES	CALLS/CASE	TOKENS/CASE	REF \$/CASE	P50	P95	SHARE OF SPEND
FAST	29	0.66	991	\$0.0008	23.8s	53.4s	5%
OBVIOUS	8	1.00	3,286	\$0.0016	21.5s	26.5s	3%
DEEP	13	4.77	14,073	\$0.0306	63.2s	83.5s	91%
Queue total			238,001	\$0.435	89 calls · free tier actual cost \$0		

Reference dollars use a configurable price table (fast 0.25/1.25, strong 3/15 per 1M tokens) to show what this call pattern would cost on paid models. The free tier itself costs nothing.

Crew versus single-judge baseline

The original single-LLM implementation is retained as a runtime mode (`AGENT_MODE=single`) so the two can be compared on identical inputs.

METRIC	SINGLE JUDGE	CREW	READING
Calls	92	89	Comparable
Tokens	327,786	238,001	-27% — routing beats brute force
Reference cost	\$0.123	\$0.435	3.5x higher — the strong-tier verifier
Band agreement		100%	Same risk bands on all 50
Action agreement		84%	8 cases differ, each surfaced for review

STATED HONESTLY

The crew is **not cheaper in dollars**. It uses fewer tokens but spends them on a more expensive model where it matters. What the extra \$0.31 across 50 cases buys is an adversarial pass, a panel on contested cases, per-domain specialisation and a complete trace. On a queue of engine referrals that is a good trade; presenting it as a pure cost win would not be true.

Weaknesses my own evaluation found

- **The challenger is under-discriminating.** It rates plausibility "medium" on all 13 DEEP cases. It still changes outcomes — it downgrades 38% of the time — but its confidence signal carries no information.
- **Confidence barely separates.** Mean self-confidence is 0.77 on contested cases versus 0.74 on uncontested. The ordering is right but the gap is too small to route on.
- **Panel agreement is high** (0.92 mean, 1 split of 5), which may mean the personas are insufficiently distinct rather than that the cases are easy.
- **One assessment carries an unverified figure flag**, which is the guardrail working, but it is a real instance of prose drifting from the record.

6 · Alternatives I rejected

OPTION	WHY NOT
An agent framework LangGraph, CrewAI, AutoGen	The interesting engineering in this problem is the control flow: budgets, typed outputs, repair, tier escalation, caching by evidence hash, fail-closed behaviour. A framework would have hidden exactly the part worth demonstrating, and its abstractions would have to be fought to enforce a hard ± 20 clamp. The harness is a few hundred lines and every behaviour is inspectable.
RAG / vector store	Fifty structured rows. SQL answers precisely; embeddings would add an approximation layer and a new failure mode to retrieve facts that are already addressable by key. This decision reverses at scale — see §8.6.
Supervised ML	No labels. The model would have to learn from targets I invented, and reporting its accuracy would be reporting how well it reproduced my own heuristic.
One LLM call per case	Built first, kept as the A/B baseline. It uses 38% more tokens for identical risk bands, and offers no adversarial pass, no specialisation and a much thinner trace.
An LLM router	Adds a call, latency and a failure mode to every case to decide something the rules already determine for free and deterministically.
Letting the model produce the score	Non-reproducible and unauditable. A bounded adjustment on a deterministic base keeps both properties while still letting evidence move the answer.
A graph database for linkage	Correct shape, wrong scale. At 50 rows an in-process join is simpler and faster; the interface is kept narrow so the store can be swapped later.

7 · With more time

In priority order, with the first item well ahead of the rest.

1 · A false-negative safety net

This is the asymmetric risk and the largest genuine gap. A false positive costs an investigator an hour; a false negative pays a fraudulent claim and never appears in any metric. Every guardrail in the system prevents the AI from clearing something risky — none of them can find what the rules never flagged. Four components, in order:

- **Miss-rate tracking.** The `outcomes` table already records whether a confirmed fraud was missed by the system and at what level it had been rated. Turn that into a standing metric with a trend, per segment and per rule, so the blind spot becomes visible.
- **Retroactive look-back.** When a case is confirmed as fraud, R17 flags its neighbours — but cases already cleared are not revisited. A look-back job should reopen cleared lookalikes and put them back in the queue with an explanation.
- **An independent adversarial reviewer.** A second pass over the `cleared` pile, optimised for recall rather than precision, whose only job is to find what was dismissed too quickly.
- **Stratified audit sampling.** Sample cleared cases by stratum and re-review them, giving a statistical bound on the miss rate rather than an anecdote.

Prototyped and deliberately deferred: a statistical anomaly detector for the silent cases. At 50 cases the set it flagged changed with the random seed (0–3 of the 29 clean cases), and a detector that unstable erodes trust faster than it finds fraud. It needs either far more data or a fundamentally more stable method.

2 · Learn from outcomes

Thresholds are currently fitted once against the portfolio distribution. With confirmed outcomes accumulating, refit domain weights and thresholds on real labels — turning the calibration story from "derived from the distribution" into "derived from what actually proved to be fraud". This is also the point at which a supervised model becomes honest.

3 · Close the evaluated weaknesses

Re-specify the challenger so its plausibility rating discriminates (few-shot anchors, or a forced ranking against sibling cases). Calibrate confidence against outcomes so it can be routed on. Widen the panel personas or replace them with genuinely different evidence views.

4 · Data and workflow depth

A provider and member graph once identifiers exist, with entity resolution. A labelled backtest replacing synthetic labels. Four-eyes approval on large bulk clears. A streaming UI so long assessments render progressively.

8 · Scaling to production volume

The prototype triages 50 cases. The question that matters is what changes at 10,000 or 100,000 referrals a day. The answer divides cleanly into what the design already gets right, what breaks first, and what the target architecture looks like.

8.1 What already scales

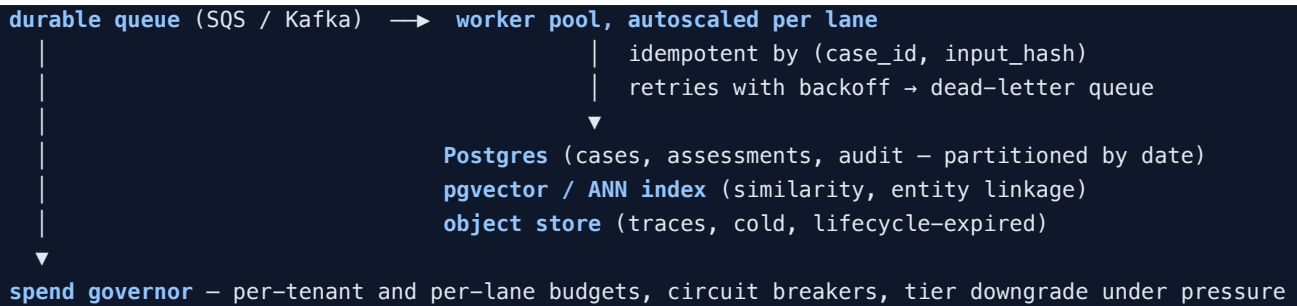
- **Routing is $O(1)$ per case and costs nothing.** The expensive path is entered only by cases that earn it. This is the single most important scaling property, and it improves with volume: a real referral queue has a **larger** clean share than this sample's 58%, so the FAST lane grows and the cost per case falls.
- **Batching.** FAST cases already share one call per five. The batch size is a tunable, not a constant.
- **Caching by evidence hash.** Re-assessment cost is proportional to what actually changed, not to case count.
- **Model tiering.** Only the verifier uses the expensive tier, and only on cases that reach it.
- **Append-only assessments with input hashes.** Idempotency and replay are already possible.
- **Fail-closed behaviour.** A degraded model provider produces visibly unassessed cases, not silent bad data — which is the correct failure mode for a system feeding human decisions.

8.2 What breaks first

COMPONENT	LIMIT	BREAKS AT ROUGHLY
SQLite single writer	WAL allows concurrent readers but serialises writes. Parallel agent workers writing spans and assessments will contend.	Low thousands of cases/day, or ~10 concurrent workers
$O(n^2)$ neighbour threshold	<code>related.nn_threshold()</code> computes every case's nearest neighbour against every other case to find the median distance. At 50 cases that is 2,500 comparisons; at 100,000 it is 10^{10} .	~10,000 cases
In-memory vector cache	<code>related._vectors()</code> loads every case's feature vector into process memory.	Hundreds of thousands of cases
In-process workers	Assessment runs as an async task inside the web process with a concurrency semaphore of 3. A restart loses in-flight work.	Immediately, for any real SLA
N+1 queries	Queue listing fetches the latest assessment per case individually.	~1,000 cases per page load
Provider rate limits	18 requests/minute on the free tier caps the whole system at ~1,000 calls/hour.	~500 cases/hour
Trace storage	Full spans and blackboards per case, unbounded retention.	~100 GB/year at 10,000 cases/day

8.3 Target architecture at volume

```
ingestion (batch or stream) → feature store → rules + score → router
|
|
| enqueue by lane, with priority = exposure × risk
▼
```



Concretely, the changes are:

- **Postgres** in place of SQLite, with `assessments`, `spans` and `audit` partitioned by date and old partitions rolled to cold storage. Read replicas for the queue and briefing views.
- **A durable queue and stateless workers** (SQS or Kafka + autoscaled consumers), one consumer group per lane so DEEP capacity scales independently of FAST. Jobs keyed on `(case_id, input_hash)` for idempotency, with retry and a dead-letter queue for poison cases.
- **An ANN index** (pgvector, FAISS or similar) replacing the exhaustive similarity scan, with the median nearest-neighbour threshold estimated from a sample rather than computed exactly. This is the point at which the "no vector store" decision reverses — not for retrieving facts, but for finding neighbours.
- **A materialised briefing.** Queue statistics, clusters and abnormal firing should be computed incrementally on ingest, not on page load.
- **A spend governor** promoted to a first-class component: per-tenant and per-lane budgets, automatic tier downgrade when a budget is nearly exhausted, and circuit breakers per provider. Cost is a production risk, not an afterthought.
- **Provider redundancy.** Two model vendors behind the tier abstraction, so a provider outage degrades throughput rather than stopping triage.

8.4 Cost model

Extrapolating the measured per-lane figures. Two mixes are shown: this sample's (58/16/26) and a more realistic production mix where the benign share is higher (80/5/15).

VOLUME	CALLS/DAY REAL MIX	TOKENS/DAY	COST/DAY SAMPLE MIX	COST/DAY REAL MIX	COST/YEAR REAL MIX
1,000 cases/day	1,294	3.1M	\$8.70	\$5.30	\$1,935
10,000 cases/day	12,935	30.7M	\$87	\$53	\$19,345
100,000 cases/day	129,350	306.8M	\$870	\$530	\$193,450

WHERE THE MONEY GOES, AND THE LEVER THAT MATTERS

DEEP is 26% of cases and **91% of spend**. Every meaningful cost lever therefore acts on the DEEP lane, not on the other 74% of the queue:

- **Tighten the routing boundary.** A one-point shift in what counts as ambiguous moves cases between a \$0.031 path and a \$0.0008 path — a 38× difference. This is worth tuning against outcomes.
- **Make the panel rarer.** It fires on 5 of 13 DEEP cases today. Better-calibrated confidence (\$7.3) would let it fire only where it changes the answer.
- **Cache harder.** Specialist reuse already exists; at volume, near-duplicate referrals from the same provider should share specialist findings outright.
- **Right-size the verifier.** It is the only strong-tier call and the reason reference cost is 3.5× the single-judge baseline. Whether it needs the strong tier on **every** DEEP case is an empirical question I have not answered.

For context, the entire AI cost at 10,000 cases a day is roughly \$53 — materially less than one investigator-hour. The economic case does not rest on model cost; it rests on how many benign cases never reach a human.

8.5 Throughput and latency

Triage is a batch workload with an overnight window, not an interactive one. Sizing it for 10,000 cases/day at the realistic mix, against a four-hour window:

LANE	CASES/DAY	UNIT OF WORK	SEQUENTIAL TIME	WORKERS FOR A 4H WINDOW
FAST (80%)	8,000	1,600 batched calls @ 24s	10.7 h	3
OBVIOUS (5%)	500	500 calls @ 21.5s	3.0 h	1
DEEP (15%)	1,500	1,500 crews @ 63s	26.3 h	7
Total				11 workers (≈14 with headroom)

That implies a sustained ~54 model calls per minute — trivial on a paid tier, and three times the free tier's entire ceiling. The binding constraint at volume is **provider rate limit and concurrency**, not compute: the workers are almost entirely waiting on network I/O, so they are cheap to run and should be scaled by queue depth rather than CPU.

Two SLAs are worth separating. **Batch triage** must finish before the morning shift — measured as queue completion time, alarmed when projected completion crosses the shift start. **Interactive actions** (chat, note re-assessment, escalation drafting) happen while an investigator waits, and need a much tighter budget: fast tier only, a hard timeout, and a partial answer rather than a spinner. The prototype's ~20s interactive calls are acceptable for a demo and would not be acceptable in production.

8.6 Data and entity resolution

The largest accuracy gain at volume is not a better model — it is better keys. With provider and member identifiers, the linkage that currently runs on claim number extends to a genuine graph: claims per provider over time, members shared between providers, contact details shared across supposedly unrelated parties, and rings that are invisible case-by-case. This needs entity resolution (fuzzy name and address matching, shared phone or bank details) as a pipeline stage, because the signal lives in identifiers that are deliberately not identical.

Peer comparison also sharpens. "Amount vs peer average" is currently care-type-relative because that is all the data supports; at volume it should be relative to care type × geography × acuity, computed on a rolling window. Portfolio thresholds likewise become rolling rather than static — which is both more accurate and a new failure mode, since a slow drift in fraudulent behaviour would drag the baseline with it. That is why thresholds should be refitted on **confirmed outcomes**, not on the referral distribution alone.

8.7 Calibration, drift and labels

- **Capture labels deliberately.** The outcome feedback loop is the most valuable thing in the system at volume, because it is the only source of ground truth. Investigator workflow should make recording an outcome effortless and close to mandatory at case closure.
- **Recalibrate on a schedule** (monthly, on a rolling window), with the new thresholds run in shadow first and compared on the impact preview the Rules tab already produces.
- **Monitor drift explicitly** — signal distributions, rule fire rates by segment, band mix, and AI-versus-rule disagreement rate. A sudden change in fire rate is either a data pipeline break or a change in fraud behaviour, and both need a human to look.
- **Version prompts and models like code.** Every assessment already stores its model; prompt version should be stored alongside it, so a quality regression can be traced to a change.
- **Champion/challenger.** The `AGENT_MODE` switch already proves the pattern. At volume, route a small percentage of traffic to a candidate configuration and compare on agreement and, once labels exist, on caught fraud.

8.8 Reliability

- **Idempotency** on `(case_id, input_hash)`, so retries and replays cannot double-write.
- **Dead-letter queue** for cases that fail repeatedly, surfaced in the UI as unassessed rather than hidden.
- **Graceful degradation ladder:** strong tier → fast tier → rules-only with an explicit "AI unavailable" state. The system is already built to show that state honestly, which makes the ladder safe.
- **Backpressure.** When the queue grows faster than it drains, downgrade lanes (DEEP → OBVIOUS treatment) rather than falling behind indefinitely, and make the downgrade visible on the affected cases.
- **Replay.** The cassette mechanism should become a production capability: replay a day's traffic against a new prompt or model and diff the outcomes before shipping.

8.9 Governance and fairness

- **Fairness is a live risk, not a theoretical one.** Distance and weekend-billing signals can proxy for rural providers and for shift-work care patterns. At volume this should be monitored as a distribution of outcomes by geography and provider type, with a periodic review of whether a rule is detecting fraud or detecting rurality.
- **Model and rule documentation.** Each rule already carries its calibration evidence; that should extend to a reviewable record for regulators and internal audit, including what the AI is permitted to do.
- **Retention and PII.** Real data is not synthetic: field-level encryption, redaction before anything leaves for a model provider, a retention policy on traces, and a documented data-processing agreement with the provider.
- **Authority split scales up.** The investigator/supervisor separation becomes real RBAC with SSO, and four-eyes approval on high-impact bulk actions.

- **The audit log becomes the compliance artefact.** It already records every AI action and human decision with a reason; at volume it needs to be tamper-evident and exportable.

8.10 Rollout

I would not switch a fraud team onto this in one step. Three phases:

1. **Shadow.** Run alongside the existing process on live referrals. Investigators work as they do today; the system's assessments are recorded but not shown. Measure agreement, and measure how often the system would have cleared something the investigator escalated — that number is the go/no-go.
2. **Assist.** Show assessments, allow accept/reject and notes, but keep bulk clear disabled. The value here is the briefing and the evidence assembly. Measure time-per-case and investigator trust directly.
3. **Act.** Enable bulk clear on the lowest band only, with four-eyes approval and mandatory sampling of cleared cases. Widen the band only as the miss-rate metric from §7.1 earns it.

Throughout, the metric that matters is not model accuracy. It is **investigator hours returned per confirmed fraud missed** — and the miss side of that ratio must be measured before the hours side is claimed.

Appendix A · Rule catalogue

All thresholds computed from the 50-case portfolio by natural-break calibration. Regenerate with `python -m app.calibration`.

ID	RULE	DOMAIN	LOGIC	FIRES
R01	Duplicate service billed	Billing	binary flag → HIGH	13/50
R02	Service overlaps another provider	Geography	binary flag → HIGH	10/50
R03	Shared contact info with provider	Relationship	binary flag → HIGH	12/50
R04	Recent policy change	Timing	binary flag → LOW (weak alone)	12/50
R05	Weekly visit frequency	Utilization	≥5.5 MEDIUM, ≥10 HIGH	18/50
R06	Member-provider distance	Geography	≥47 mi MEDIUM, ≥160 mi HIGH	14/50
R07	Amount vs peer average	Billing	≥16% MEDIUM, ≥76% HIGH	18/50
R08	Round-dollar billing ratio	Billing	≥0.29 MEDIUM, ≥0.62 HIGH	18/50
R09	Weekend billing ratio	Billing	≥0.17 MEDIUM, ≥0.42 HIGH	19/50
R10	Prior claims (12 mo)	Utilization	≥2.5 MEDIUM, ≥7 HIGH	17/50
R11	Implied weekly travel miles (derived)	Geography	≥1,800 HIGH, ≥4,510 CRITICAL	8/50
R12	High-value claim (derived)	Billing	log-amount robust z ≥0.72 MEDIUM; HIGH above p90 (\$32,645)	15/50
R13	Multi-domain convergence (derived)	Convergence	≥3 domains HIGH, ≥4 CRITICAL	16/50
R14	Silent drift (derived)	Convergence	0 flags and ≥3 elevated continuous signals → MEDIUM	3/50
R15	Claim number reused across cases	Integrity	same claim number on >1 referral → HIGH	2/50
R16	High-value claim after policy change	Timing	policy change AND amount ≥ p75 (\$13,131) → MEDIUM	10/50
R17	Lookalike of a confirmed-fraud case	Precedent	within median nearest-neighbour distance of a human-confirmed fraud: 1 → MEDIUM, 2+ → HIGH; never bulk-clearable	0/50

Documented gaps — signals the data cannot support: days since policy change (only a 0/1 flag), provider identity, member identity, diagnosis.

Appendix B · Repository map

PATH	CONTENTS
<code>app/main.py</code>	FastAPI application: cases, assessment, chat, bulk actions, escalation, rules, blocklist, briefing, audit, runs. Serves the SPA with cache-busted assets.
<code>app/service.py</code>	Ingest, scoring, notes with full purge, outcomes, bulk evaluation, escalation, blocklist application and override, input hashing.
<code>app/rules.py</code> · <code>calibration.py</code> · <code>features.py</code> · <code>scoring.py</code>	The deterministic engine: feature derivation, natural-break calibration, rule registry, domain-capped noisy-OR scoring.
<code>app/agents/</code>	<code>router.py</code> (lane selection), <code>crew.py</code> (specialists, challenger, verifier, panel), <code>roles.py</code> (prompts and the shared grounding contract), <code>runtime.py</code> (budgets, repair, tiers, spans), <code>evidence.py</code> , <code>schemas.py</code> , <code>domains.py</code> .
<code>app/llm/</code>	<code>client.py</code> (tiers, rate limiter, circuit breaker, fallback, cassette), <code>judge.py</code> (orchestration and batching), <code>guardrails.py</code> (the verification boundary), <code>tools.py</code> (9 read-only tools), <code>chat.py</code> , <code>rule_draft.py</code> .
<code>app/evals/</code>	<code>invariants.py</code> , <code>metrics.py</code> , <code>perturb.py</code> (counterfactual and injection), <code>run.py</code> .
<code>app/backtest.py</code> · <code>rulestore.py</code> · <code>customrules.py</code> · <code>blocklist.py</code> · <code>related.py</code> · <code>clusters.py</code> · <code>briefing.py</code>	Governance surfaces, cross-case knowledge, queue-level analytics.
<code>web/</code>	Single-page app, no build step: Briefing, Queue, Case, Rules, Blocklist, Supervisor, Audit.
<code>tests/</code>	88 tests across rules and scoring, guardrails, the harness, the router, notes and outcomes, the Rules tab, the blocklist, and bulk selection.
<code>docs/</code>	<code>WRITEUP.md</code> , <code>Writeup.pptx</code> (the 4-slide deck), <code>RULES.md</code> (auto-generated calibration evidence), <code>EVAL.md</code> (auto-generated eval report), architecture diagrams and their generators.
<code>Dockerfile</code> · <code>render.yaml</code>	Container build and deployment blueprint, with a persistent data volume and an optional shared-password gate.

Running it

`pip install -r requirements.txt` · `set OPENROUTER_API_KEY` in `.env` · `python -m uvicorn app.main:app --port 8000` · open `http://localhost:8000`. The 50 cases ingest and score on startup and AI triage begins automatically. Evaluation: `python -m app.evals.run --perturb --shadow`. Tests: `pytest -q`.